Project Exposé and Work Plan

Imitation-Reinforcement Learning for Dynamic Multi-Period Mean-Variance Portfolio Optimisation

Arkan Alexei Andrei
Nick Bermbach

Supervisor: Ardalan Azarnejad
Chair of Financial Economics and Risk Management (C-RAM)
Prof. Dr. Maxim Ulrich
Karlsruhe Institute of Technology

June 12, 2026

Abstract

This project investigates a hybrid imitation–reinforcement learning (IL–RL) approach for dynamic, multi-period mean–variance portfolio allocation in a controlled regime-switching market environment. A rolling-horizon Markowitz optimiser with transaction costs and portfolio constraints serves as an economically interpretable expert demonstrator. The learning pipeline is designed to reproduce the expert's state-to-allocation mapping efficiently and to examine whether reinforcement learning fine-tuning can improve realised out-of-sample utility when decisions are based on estimated moments and filtered regime information. The complete project scope comprises: regime-switching return simulation, rolling-horizon quadratic-programming expert decisions, expert trajectory generation, a transformer-diffusion policy trained sequentially through behavioural cloning, Dataset Aggregation (DAgger), Wasserstein-regularised Generative Adversarial Imitation Learning (GAIL), Proximal Policy Optimization (PPO) fine-tuning, a formal regret analysis under covariance-estimation error and regime shifts, and a final evaluation over 500 independent Monte Carlo runs. The exposé formulates the expert without look-ahead information, defines a reward that charges transaction costs exactly once, and states theoretical guarantees conditionally: estimation error may yield an unavoidable linear residual term unless it vanishes sufficiently quickly. The project is planned over two semesters, approximately ten months.

Contents

1 Motivation **3**

2 Research Aim **4**

2.1 Research Questions 4

3 Problem Setting and Information Structure **5**

3.1 Market Environment 5

3.2 Decision Timing and No-Look-Ahead Principle 5

4 Methodology **6**

4.1 Return Simulation and Regime Filtering 6

4.2 Rolling-Horizon Markowitz Expert 6

4.3 State and Action Representation 7

4.4 Transformer-Diffusion Allocation Policy 7

4.5 Imitation Learning Pipeline 8

4.6 Reinforcement Learning Fine-Tuning 8

4.7 Theoretical Regret Analysis 9

5 Experimental Design and Evaluation Strategy **10**

5.1 Data Splits and Generalisation Protocol 10

5.2 Strategies and Ablations 10

5.3 Performance Metrics 11

5.4 Statistical Inference 11

6 Expected Contributions and Limitations **12**

7 Work Plan: Two-Semester Schedule **13**

8 Conclusion **14**

1 Motivation

Portfolio selection deals with the question of how capital should be distributed among different assets when future returns are uncertain. In the classical mean–variance framework introduced by Markowitz [1952], portfolio weights are chosen by considering both expected return and risk, measured through variance. This framework is still widely used as a benchmark because it provides a clear and interpretable way of representing diversification and risk preferences. However, the basic model describes a single investment decision and does not directly account for several aspects that are relevant in practice. For example, market conditions can change over time, portfolio adjustments lead to transaction costs, estimates of expected returns and covariances are uncertain, and constraints may restrict the possible allocations [Kolm et al., 2014].

These issues become more important in a dynamic setting, where portfolio decisions are made repeatedly over time. At each rebalancing date, the investor must choose a new allocation based only on the information available at that point. A rolling-horizon Markowitz strategy that includes transaction costs and portfolio constraints can serve as a meaningful expert benchmark for this setting. At the same time, such a strategy requires solving an optimisation problem again at every decision point. This may become computationally demanding when many assets, long simulation periods, and a large number of Monte Carlo runs are considered. In addition, the resulting allocations may depend strongly on estimated means and covariance matrices.

This project therefore investigates whether a learned policy can approximate the decisions of an optimisation-based expert. The idea is not to replace established financial theory with a purely data-driven model. Instead, imitation learning is used to train an agent on portfolio decisions generated by the rolling-horizon Markowitz expert. In a second step, reinforcement learning can be used to fine-tune the policy while considering realised utility, transaction costs, and portfolio constraints. This makes it possible to study whether a learned policy can reproduce expert decisions more efficiently and whether it can adapt when market regimes or estimated moments change.

2 Research Aim

The aim of the project is to develop and evaluate a reproducible hybrid IL–RL policy for dynamic portfolio allocation that learns from a friction-aware, rolling-horizon Markowitz expert in a simulated regime-switching financial environment. The learned policy will be assessed with respect to expert imitation, economic performance, robustness under distribution shifts, and decision-time computation.

2.1 Research Questions

The study addresses the following research questions:

1. How numerically stable and reproducible is a rolling-horizon, friction-aware Markowitz expert under latent regime shifts, estimated moments, and portfolio constraints?
2. To what extent can behavioural cloning, DAgger, and Wasserstein-regularised GAIL reproduce the expert's allocation decisions and trajectory-level behaviour?
3. Does the transformer-diffusion policy offer advantages relative to a deterministic baseline policy in allocation quality, turnover, robustness, or generalisation to unseen simulated regimes and covariance structures?
4. After imitation warm-starting, can PPO fine-tuning improve realised out-of-sample certainty-equivalent utility without unacceptable turnover, drawdown, or deviation from the expert benchmark?
5. What inference-time benefit does the learned policy offer relative to resolving the rolling-horizon quadratic program at every rebalancing date?
6. Under which assumptions can cumulative performance loss relative to the expert be bounded when covariance estimates are imperfect and the market undergoes regime shifts?

3 Problem Setting and Information Structure

3.1 Market Environment

Let N denote the number of risky assets and let $r_{t+1} \in \mathbb{R}^N$ denote the vector of returns realised after the allocation decision at time t. Market dynamics are governed by a latent regime variable $z_t \in \{1, \dots, K\}$ evolving according to a Markov chain with transition matrix P:

$$\Pr(z_{t+1} = j \mid z_t = i) = P_{ij}. \tag{1}$$

Conditional on the next regime, daily returns follow a multivariate Gaussian distribution,

$$r_{t+1} \mid z_{t+1} = k \sim \mathcal{N}\left(\mu^{(k)}, \Sigma^{(k)}\right), \qquad k \in \{1, \dots, K\}. \tag{2}$$

The use of three regimes—for example calm, volatile, and stressed regimes—creates a controlled non-stationary data-generating process suitable for studying changing means, covariance structures, and transaction-cost-sensitive allocations. Markov-switching specifications are grounded in established time-series modelling of regime changes [Hamilton, 1989].

The final simulation study follows the scale specified in the project brief: $N \in \{50, 100, 200\}$ assets and paths of length $T = 10{,}000$ daily observations. Smaller configurations are used during unit testing, debugging, and architecture tuning but are not substitutes for the final-scale experiments. Transaction costs are varied within the prescribed range $c \in [0.001, 0.005]$.

3.2 Decision Timing and No-Look-Ahead Principle

At time t, the policy has observed returns only up to r_t and knows previously executed weights. Define the available information set by

$$\mathcal{F}_t = \sigma(r_1, \dots, r_t,\, w_0, \dots, w_{t-1})\,. \tag{3}$$

The decision w_t is required to be $\mathcal{F}_t$-measurable and is held over the next return interval, generating realised gross portfolio return $w_t^\top r_{t+1}$. The latent simulated regime z_t is available to the simulator for evaluation, but it is not supplied directly to the main policy or expert estimation procedure. Instead, a filtering model estimates regime probabilities from past observed returns.

This timing convention is essential for avoiding look-ahead bias. In particular, multi-period forecasts used by the expert are denoted by $\hat{\mu}_{\tau+1|t}$ and $\hat{\Sigma}_{\tau+1|t}$ to emphasize that they are formed at time t rather than observed from future simulated data.

4 Methodology

4.1 Return Simulation and Regime Filtering

The project uses simulated daily stock returns so that different market conditions can be studied in a controlled setting. For each simulation run, regime-specific mean vectors and positive-semidefinite covariance matrices are defined or sampled from suitable parameter ranges. A sequence of market regimes is then generated using a Markov transition model, and daily returns are sampled according to Equation (2). Basic diagnostic checks will be carried out to examine whether the simulated data reflect the intended regime frequencies, persistence, return means, variances, and covariance structures.

In practice, the current market regime is not directly observable. For this reason, the policy will mainly use estimated regime probabilities,

$$\hat{p}_t(k) = \Pr(z_t = k \mid r_1, \ldots, r_t), \qquad k = 1, \ldots, K, \tag{4}$$

which are obtained from a Hidden Markov Model using only information available up to time t [Hamilton, 1989]. For one-step-ahead forecasts, the filtered probabilities are propagated through the transition matrix,

$$\Pr(z_{t+1} = k \mid \mathcal{F}_t) = \sum_{i=1}^{K} \hat{p}_t(i) P_{ik},$$

before constructing $\hat{\mu}_{t+1|t}$ and $\hat{\Sigma}_{t+1|t}$. Expected returns and covariance matrices will be estimated using rolling windows. Since covariance estimation becomes more difficult when the number of assets is large, shrinkage covariance estimators will also be considered [Ledoit and Wolf, 2004].

4.2 Rolling-Horizon Markowitz Expert

The expert benchmark is based on a rolling-horizon mean–variance optimisation problem, building on the classical Markowitz framework and practical extensions with transaction costs and portfolio constraints [Markowitz, 1952, Kolm et al., 2014]. The main setting uses a planning horizon of $H = 21$ trading days. At each decision date t, the expert estimates future returns and covariance matrices using only the information available at that time, denoted by $\mathcal{F}_t$, together with the filtered regime probabilities. The resulting optimisation problem is

$$\begin{aligned} \{w^*_{\tau|t}\}_{\tau=t}^{t+H-1} = \arg\max_{\{w_{\tau|t}\}} \quad & \sum_{\tau=t}^{t+H-1} \left[w_{\tau|t}^\top \hat{\mu}_{\tau+1|t} - \frac{\gamma}{2} w_{\tau|t}^\top \hat{\Sigma}_{\tau+1|t} w_{\tau|t} - c \left\| w_{\tau|t} - w_{\tau-1|t} \right\|_1 \right] \\ \text{s.t.} \quad & \mathbf{1}^\top w_{\tau|t} = 1, \\ & \ell_i \le w_{i,\tau|t} \le u_i, \qquad i = 1, \ldots, N, \\ & \left\| w_{\tau|t} \right\|_1 \le L, \qquad \tau = t, \ldots, t+H-1. \end{aligned} \tag{5}$$

For $\tau = t$, $w_{\tau-1|t}$ is set equal to the previously implemented portfolio w_{t-1}. For later planning steps, $w_{\tau-1|t}$ denotes the preceding planned portfolio within the horizon.

Here, $\gamma > 0$ represents risk aversion, c is the proportional transaction-cost parameter, (ℓ_i, u_i) defines the lower and upper bounds for individual asset weights, and L is a leverage limit. The experiments will use a long-only portfolio setting, so that $\ell_i = 0$. In the long-only baseline, the leverage constraint is redundant because $\left\| w_{\tau|t} \right\|_1 = 1$; it is retained in the formulation to allow later extensions with limited short positions.

Although the expert plans several periods ahead, only the first portfolio decision,

$$w^*_t = w^*_{t|t},$$

is implemented. At the next date, the optimisation problem is solved again using the newly available information. In this way, the expert adapts its decisions over time rather than fixing an allocation plan in advance.

The optimisation problem will be implemented using `CVXPY`, which allows convex optimisation problems to be formulated directly in Python [Diamond and Boyd, 2016]. Before the expert is used to generate training data, the implementation will be checked for feasible portfolio weights, correct solver behaviour, reasonable runtime, and numerical stability under different parameter settings.

4.3 State and Action Representation

The expert generates pairs of states and portfolio decisions, denoted by (s_t, w_t^*). The state at time t is constructed as

$$s_t = \left[r_{t-L+1:t},\ \hat{\mu}_{t+1|t},\ \mathrm{vech}(\hat{\Sigma}_{t+1|t}),\ \hat{p}_t,\ w_{t-1},\ c,\ \gamma \right], \tag{6}$$

where L denotes the return lookback window. The state therefore contains recent returns, estimated moments, estimated regime probabilities, the previous portfolio allocation, and the main economic parameters. The operator $\mathrm{vech}(\cdot)$ stores the lower triangular part of the symmetric covariance matrix in vector form.

The action of the policy is the portfolio allocation,

$$a_t = w_t \in \Delta^{N-1} = \{ w \in \mathbb{R}^N : \mathbf{1}^\top w = 1,\ w_i \geq 0 \}. \tag{7}$$

Thus, in the main long-only setting, the policy must always produce non-negative asset weights that sum to one.

One practical challenge is that the state vector can become very large. For example, the covariance matrix alone contributes $N(N+1)/2$ entries. With $N = 50$ assets and a lookback window of $L = 20$, a direct state representation already contains 2,380 numerical inputs. This becomes substantially larger when the asset universe is expanded. The main approach will therefore use the transformer encoder to process the return history and combine it with estimated moments, regime probabilities, previous weights, and economic parameters. If needed, more compact state representations, such as shrinkage- or factor-based covariance summaries, may be considered as practical scalability options.

4.4 Transformer-Diffusion Allocation Policy

The planned policy model combines a transformer-based state encoder with a conditional diffusion model for portfolio weights. The transformer is used to process the sequence of historical returns together with additional state information. This follows the attention-based sequence modelling approach introduced by Vaswani et al. [2017]. Based on the encoded state, the diffusion component generates a portfolio allocation through a sequence of denoising steps. Diffusion models provide a flexible way of modelling continuous outputs [Ho et al., 2020], and related diffusion-policy approaches have been applied in sequential decision problems [Chi et al., 2023].

Since a portfolio allocation must satisfy the investment constraints, the generated action cannot be used without checking feasibility. During the reverse diffusion process, each provisional portfolio vector is therefore projected onto the simplex:

$$\tilde{w}_t^{(j-1)} = f_\theta\left(\tilde{w}_t^{(j)}, s_t, j\right), \qquad w_t^{(j-1)} = \Pi_{\Delta^{N-1}}\left(\tilde{w}_t^{(j-1)}\right), \tag{8}$$

where j denotes the reverse diffusion step and $\Pi_{\Delta^{N-1}}(\cdot)$ denotes the Euclidean projection onto the long-only portfolio simplex, ensuring that the generated weights are non-negative and sum to one.

4.5 Imitation Learning Pipeline

The imitation-learning stage is divided into three parts. Each stage builds on the previous one and allows the policy to be compared with the expert benchmark in a more demanding setting.

Stage 1: Behavioural cloning. The first step treats expert imitation as a supervised learning problem, following the standard behavioural cloning setup discussed in imitation learning [Ross et al., 2011]. Using portfolio decisions generated by the Markowitz expert, the demonstration dataset is defined as

$$\mathcal{D}_E = \{(s_t, w_t^*)\}_{t=1}^{M}.$$

The policy is trained to predict the expert's allocation from the corresponding state. Since the action is a continuous portfolio-weight vector, a natural baseline objective is the mean squared allocation error,

$$L_{\mathrm{BC}}(\theta) = \frac{1}{M}\sum_{t=1}^{M} \|\pi_\theta(s_t) - w_t^*\|_2^2 + \lambda_\theta \Omega(\theta),$$

where the first term measures the distance between the learned and expert allocations, while the second term represents regularisation. In addition to this basic loss, the experiments may consider alternatives that are more closely related to portfolio performance, such as utility loss or portfolio-weight divergence.

Stage 2: Dataset Aggregation (DAgger). Behavioural cloning learns only from states observed in the original expert demonstrations. However, once the learned policy makes its own decisions, small errors can lead it to states that were not well represented in the original training data. This issue is especially relevant in dynamic portfolio allocation because the current allocation influences future transaction costs and therefore future decisions.

DAgger addresses this problem by letting the current policy interact with the simulated environment, asking the expert what it would have done in the visited states, and adding these new state–action pairs to the training data [Ross et al., 2011]. The policy is then retrained using the expanded dataset. This allows the project to investigate whether collecting examples from the learner's own trajectories improves its ability to follow the expert over time.

Stage 3: Wasserstein-Regularised GAIL. The third imitation-learning stage considers GAIL, which aims to match the overall behaviour of the expert rather than only predicting individual expert actions [Ho and Ermon, 2016]. In this project, the method is extended using a Wasserstein-based critic, following ideas from Wasserstein GAN training [Arjovsky et al., 2017, Gulrajani et al., 2017]. For the expert occupancy distribution d_E and the policy occupancy distribution d_π, the Wasserstein distance is represented as

$$W_1(d_E, d_\pi) = \sup_{\|D\|_{\mathrm{Lip}} \le 1} \Big(\mathbb{E}_{(s,a)\sim d_E}[D(s,a)] - \mathbb{E}_{(s,a)\sim d_\pi}[D(s,a)] \Big). \tag{9}$$

The resulting policy will then serve as the starting point for the reinforcement-learning stage.

4.6 Reinforcement Learning Fine-Tuning

After imitation learning, the policy will be fine-tuned using reinforcement learning. The aim is to examine whether a policy that has already learned from the expert can improve its realised portfolio performance while still remaining close to economically reasonable decisions.

The realised net portfolio return after choosing allocation w_t is defined as

$$\tilde{R}_{t+1}(w_t) = w_t^\top r_{t+1} - c\,\|w_t - w_{t-1}\|_1 . \tag{10}$$

The second term accounts for transaction costs caused by changes in the portfolio weights. These costs are included once in the realised return and will not be added again as a separate penalty in the main reward function.

The PPO reward is defined as

$$\rho_t = \tilde{R}_{t+1}(w_t) - \frac{\gamma}{2} w_t^\top \hat{\Sigma}_{t+1|t} w_t - \lambda_E \left\| w_t - w_t^* \right\|_2^2, \tag{11}$$

where the second term penalises estimated portfolio risk and the third term penalises large deviations from the expert allocation. The strength of the expert-deviation penalty is controlled by λ_E. Turnover will also be reported as an evaluation measure, but it is not added separately to the main reward because transaction costs already capture the direct cost of changing portfolio weights.

PPO is chosen because it provides a practical method for updating a policy while limiting very large changes between training steps [Schulman et al., 2017].

Any improvement over the Markowitz expert must be interpreted carefully. The expert optimises an estimated objective using imperfect information, whereas the final evaluation uses realised simulated returns. Therefore, differences in performance may result from estimation error, regime uncertainty, the planning horizon, or the way the learned policy reacts to distribution changes.

4.7 Theoretical Regret Analysis

The theoretical part of the project studies how much performance is lost when the learned policy is compared with the expert benchmark. Let $u_t(w)$ denote the one-period portfolio objective after accounting for return, risk, and transaction costs in the simulated environment. Regret relative to the expert is defined as

$$\mathrm{Reg}_T(\pi; \pi_E) = \sum_{t=1}^{T} \left[u_t(w_t^*) - u_t(w_t) \right]. \tag{12}$$

The analysis will distinguish between three possible sources of performance loss. First, the learned policy may not reproduce the expert decisions perfectly. Second, the moments used by the expert and the policy are estimated and may differ from the true simulated moments. Third, the learned policy may visit different states from the expert, which creates a difference between their state–action distributions.

Motivated by no-regret imitation-learning arguments, occupancy-measure matching in GAIL, and Wasserstein-based distributional discrepancies [Ross et al., 2011, Ho and Ermon, 2016, Arjovsky et al., 2017], a possible regret decomposition to be studied is

$$\mathrm{Reg}_T(\pi; \pi_E) \leq C_1 \sqrt{T \log N} + C_2 \sum_{t=1}^{T} \epsilon_t + C_3 \sum_{t=1}^{T} W_1(d_{E,t}, d_{\pi,t}), \tag{13}$$

where ϵ_t represents estimation error in quantities such as expected returns or covariance matrices, and $W_1(d_{E,t}, d_{\pi,t})$ captures differences between the expert and policy occupancy distributions.

This decomposition is useful because it separates errors caused by learning from errors caused by uncertain information and changing market conditions. An overall sublinear regret result can only be expected if the accumulated estimation errors and distribution differences also grow slower than linearly in T. If these errors remain large over time, part of the performance loss cannot be avoided simply by improving the policy model.

5 Experimental Design and Evaluation Strategy

5.1 Data Splits and Generalisation Protocol

Each Monte Carlo run generates independent simulated market paths and associated expert trajectories. Data separation is performed at the path level whenever possible to avoid training and evaluating on overlapping stochastic realisations. Three evaluation settings are distinguished:

(i) **In-distribution holdout:** new random paths drawn from simulation parameters represented during training;

(ii) **Temporal holdout:** later segments of held-out paths, with decisions constructed solely from earlier information; and

(iii) **Distribution-shift / zero-shot evaluation:** unseen transition matrices, covariance configurations, regime persistence patterns, transaction-cost parameters, or asset-universe settings.

The final experiment consists of 500 independent Monte Carlo runs. Development runs with fewer paths are used for debugging and hyperparameter screening, after which final evaluation configurations are frozen and recorded before the large-scale comparison.

5.2 Strategies and Ablations

The evaluation will include the following main strategies:

1. equal-weight portfolio as a simple benchmark;
2. rolling-horizon Markowitz expert;
3. deterministic transformer imitation policy;
4. transformer-diffusion imitation policy;
5. hybrid imitation-initialised PPO policy;
6. pure PPO policy without imitation initialisation.

The main ablations compare pure imitation learning with imitation-initialised PPO fine-tuning, and the transformer-diffusion policy with the deterministic transformer policy. These comparisons are intended to assess the contribution of the IL warm start, the RL fine-tuning step and the diffusion-based action model.

5.3 Performance Metrics

For realised net returns $\tilde{R}_t$, the financial and imitation metrics include annualised return, annualised volatility, Sharpe ratio, Sortino ratio, maximum drawdown, turnover, certainty-equivalent utility, expert tracking error, utility loss, and decision-time latency. Representative quantities are

$$\text{Sharpe} = \sqrt{252}\,\frac{\overline{\tilde{R}} - r_f/252}{\text{sd}(\tilde{R})}, \tag{14}$$

$$\text{Turnover} = \frac{1}{T}\sum_{t=1}^{T} \|w_t - w_{t-1}\|_1\,, \tag{15}$$

$$\text{CE}(\pi) = \mathbb{E}[\tilde{R}^\pi] - \frac{\gamma}{2}\,\text{Var}(\tilde{R}^\pi), \tag{16}$$

$$\text{UtilityLoss}(\pi) = \text{CE}(\pi_E) - \text{CE}(\pi), \tag{17}$$

$$\text{TrackingError}(\pi) = \frac{1}{T}\sum_{t=1}^{T} \|w_t - w_t^*\|_2^2\,. \tag{18}$$

Inference time will be measured under documented hardware and batching conditions to ensure that comparisons between neural policies and repeated optimisation are interpretable.

5.4 Statistical Inference

For strategy-level performance measures, the main comparison will be based on paired differences across matched Monte Carlo runs. This means that all strategies are evaluated on the same simulated return paths, and their performance is compared path by path. This is useful because differences between strategies can then be interpreted more clearly, without being driven by different random market simulations.

Bootstrap confidence intervals will be used to assess how stable the observed performance differences are across simulation runs [Efron and Tibshirani, 1993]. They will be reported for differences in Sharpe ratio, certainty-equivalent utility, turnover, maximum drawdown, and inference latency.

Predictive-ability tests will only be used when a suitable sequence of per-period loss differences can be defined. For two strategies A and B, the per-period economic loss differential is given by

$$\delta_t = \ell_t^A - \ell_t^B, \qquad \ell_t^m = -\left(\tilde{R}_{t+1}(w_t^m) - \frac{\gamma}{2}(w_t^m)^\top \hat{\Sigma}_{t+1|t} w_t^m\right). \tag{19}$$

The Diebold–Mariano test can then be used to test for equal average economic loss, while the Giacomini–White test can be used to study conditional predictive ability if an appropriate conditioning specification is defined [Diebold and Mariano, 1995, Giacomini and White, 2006]. Any use of these tests will state the loss sequence, dependence correction, and tested hypothesis explicitly.

6 Expected Contributions and Limitations

The project aims to make four main contributions:

1. The project will develop a reproducible simulation environment in which relevant factors such as market regimes, uncertain covariance estimates, transaction costs, and portfolio constraints can be adjusted and tested separately.
2. A rolling-horizon Markowitz expert will be implemented and used as a benchmark. The project will then examine how well imitation-learning methods can reproduce its allocation decisions in a dynamic setting.
3. The project will explore a hybrid imitation-learning and reinforcement-learning approach, in which an imitation-learning policy is subsequently fine-tuned using PPO.
4. Finally, the project will discuss regret and performance loss by distinguishing between errors caused by the learning algorithm and errors resulting from imperfect moment estimates or changing return distributions.

The analysis is entirely based on simulated data. Therefore, the results should be interpreted as evidence about the behaviour and feasibility of the proposed methodology under controlled conditions, rather than as evidence of profitable real-world trading performance. In addition, the assumption of Gaussian, regime-dependent returns is a simplified representation of financial markets and therefore limits the extent to which the findings can be transferred directly to real data.

7 Work Plan: Two-Semester Schedule

The project is planned for two semesters, which corresponds to roughly ten months or 40 working weeks.

Table 1: Planned work schedule for the two-semester project.

Wks.	Work package	Main tasks
1–4	Literature and planning	Review the relevant literature, define the research questions and notation, decide on parameter ranges and benchmarks, and set up conventions for reproducible experiments.
5–8	Simulation environment	Implement a return simulator with changing market regimes, define three regimes, add transaction-cost parameters, and test whether the generated data behave as intended.
9–13	Markowitz expert	Implement the rolling-horizon Markowitz benchmark, including estimated moments, covariance estimation options, transaction costs, and portfolio constraints. Test whether the optimisation runs correctly and produces feasible portfolios.
14–16	Dataset and environment	Define the state and action variables, prepare training, validation, and test paths, build the environment, and test that no future information is used in the decision process.
17–21	Policy model and behavioural cloning	Implement a transformer-based policy baseline and, where feasible, the planned transformer-diffusion policy. Train the first behavioural cloning models and compare prediction quality and runtime.
22–25	DAgger training	Extend the imitation-learning stage using DAgger by collecting states visited by the learned policy, requesting expert actions for these states, and retraining the model on the expanded dataset.
26–29	GAIL extension	Investigate the planned Wasserstein-regularised GAIL extension, monitor training stability, and compare its results with behavioural cloning and DAgger.
30–33	PPO fine-tuning	Fine-tune the learned policy with PPO using the defined reward function. Compare the hybrid approach with a pure PPO baseline and check turnover, constraints, and utility results.
18–35	Theoretical analysis	Develop the theoretical discussion alongside the implementation. Analyse how imitation error, estimation error, and changing regimes affect performance, and formulate the relevant assumptions clearly.
34–37	Robustness checks	Test how results change under different transaction costs, risk-aversion settings, covariance estimators, planning horizons, numbers of assets, and regime settings. Compare different model and training-stage variants.
38–39	Final evaluation	Run the final Monte Carlo experiments using fixed configurations. Compute performance measures, confidence intervals, runtime comparisons, and prepare final figures and tables.
40	Final report and submission	Finish the written report and presentation, organise the code and documentation, check that the experiments can be reproduced, and prepare the final submission.

8 Conclusion

This project develops a complete hybrid imitation–reinforcement learning framework for dynamic multi-period mean–variance portfolio optimisation. A controlled regime-switching simulator and a no-look-ahead rolling-horizon Markowitz expert form the basis for generating economically interpretable demonstrations. A transformer-diffusion policy is trained through behavioural cloning, DAgger, and Wasserstein-regularised GAIL, then fine-tuned using PPO under a reward that accounts consistently for returns, risk, transaction costs, and expert deviation. The final analysis combines financial outcomes, expert imitation, runtime, robustness, zero-shot evaluation, statistical inference, and a theoretical regret study. The resulting contribution is intended to be a reproducible and scientifically defensible investigation of learning-based approximation and adaptation of repeated portfolio optimisation under non-stationarity and market frictions.

References

Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223. PMLR, 2017.

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems XIX*, 2023.

Steven Diamond and Stephen Boyd. CVXPY: A python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

Francis X. Diebold and Robert S. Mariano. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

Raffaella Giacomini and Halbert White. Tests of conditional predictive ability. *Econometrica*, 74 (6):1545–1578, 2006.

Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein GANs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

James D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

Petter N. Kolm, Reha Tütüncü, and Frank J. Fabozzi. 60 years of portfolio optimization: Practical challenges and current trends. *European Journal of Operational Research*, 234(2):356–371, 2014. doi: 10.1016/j.ejor.2013.10.060.

Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004. doi: 10.1016/S0047-259X(03) 00096-4.

Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952. doi: 10.1111/j. 1540-6261.1952.tb01525.x.

Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635. PMLR, 2011.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.